Numerical Methods for Differential Equations

Lab Project - Part I
Finite Element solution of elliptic equations

Dipartimento di Matematica – Università di Padova

April 17, 2026

Contents

1 Problem formulation **1**
1.1 FEM solution with Galerkin approach 1

2 The code **2**
2.1 Definition of the computational mesh 2
2.2 Evaluation of elemental matrices and their assembly into the final global matrix 4
2.3 Evaluation of the right hand side vector 6
2.4 Imposing the Dirichlet boundary conditions . . . 7
2.5 Solution of the linear system 7

3 Application of the code **7**

A The Matlab templates and the mesh definitions **8**

B Assembly of the stiffness matrix from the local contributions **12**

1 Problem formulation

Consider the Poisson equation

$$-\Delta u = f \qquad \text{in } \Omega \subset \mathbb{R}^2\,, \tag{1.1a}$$

$$u = u_0 \qquad \text{in } \Gamma_D \subset \partial\Omega \tag{1.1b}$$

$$\nabla u \cdot \nu = q_N \qquad \text{in } \Gamma_N \subset \partial\Omega \tag{1.1c}$$

where Eqs. (1.1b) and (1.1c) are the Dirichlet and Neumann boundary conditions. We want to solve this problem by means of the Galerikin $\mathcal{P}_1$ Finite Element Method (FEM). We consider the following domain:

$$\Omega = \{(x,y) : (x,y) : -1 \le x \le 1, -1 \le y \le 1\}\,,$$

the forcing function:

$$f(x,y) = -5/4\pi^2 \sin(\pi x)\cos(\pi/2\ y),$$

and the boundary conditions on Γ_D and Γ_N ($\Gamma = \Gamma_D \cup \Gamma_N = \partial\Omega$) given by:

$$\begin{aligned}
u &= 0 && \text{on } \Gamma_D = \{(x,y) : -1 \le x \le 1, y = \pm 1\} \\
\nabla u \cdot \nu = q_1 &= -\pi \cos\left(\frac{\pi}{2} y\right) && \text{on } \Gamma_{N1} = \{(x,y) : x = 1, -1 \le y \le 1\} \\
\nabla u \cdot \nu = q_2 &= \pi \cos\left(\frac{\pi}{2} y\right) && \text{on } \Gamma_{N2} = \{(x,y) : x = -1, -1 \le y \le 1\}
\end{aligned}$$

The explicit solution to this problem is given by:

$$u\,(x,y) = \sin(\pi x)\cos\left(\frac{\pi}{2} y\right) \tag{1.2}$$

1.1 FEM solution with Galerkin approach

Given a triangulation $\mathcal{T}_h(\Omega)$ of the domain formed by n nodes and M triangles, The Galerkin FEM formulation of the problem is given by:

Problem 1.1.1. Find $u_h \in \mathcal{V}_h = \{\phi_1, \phi_2, \ldots, \phi_n\}$ such that:

$$a\,(\phi_i, \phi_j) = (f, \phi_i) + (q_n, \phi_i)_{\Gamma_N}\,, \qquad i = 1, \ldots, n,$$

where

$$a\,(\phi_i, \phi_j) = \int_\Omega \nabla\,\phi_i \cdot \nabla\,\phi_j\ \mathrm{d}x; \quad (f, \phi_i) = \int_\Omega f\phi_i\ \mathrm{d}x; \quad (q_n, \phi_i)_{\Gamma_N} = \int_{\Gamma_N} q_n\phi_i\ \mathrm{d}s.$$

The symmetric and positive definite linear system of n equations and n unknowns $u_1, u_2, \ldots, u_n$ is given by:

$$Au = b \tag{1.3}$$

where A is the "stiffness matrix", $a_{ij} = a(\phi_i, \phi_j)$, and b is the load vector $b_i = (f, \phi_i) + (g, \phi_i)_{\Gamma_{N,h}}$.
We use as basis functions conforming piecewise linear two-dimensional interpolating polynomials defined element-by-element. Thus, the (i, j)-th coefficient a_{ij} of the stiffness matrix is the result of the *assembly* of the local contributions $a_{ij}^{(T)}$:

$$a_{ij} = \sum_{T \in \mathcal{T}_h(\Omega)} a_{ij}^{(T)} = \sum_{T \in \mathcal{T}_h(\Omega)} \int_T \nabla \phi_i^{(T)} \cdot \nabla \phi_j^{(T)} \, \mathrm{d}x \, \mathrm{d}y \tag{1.4}$$

while the right hand side b_i is given by:

$$b_i = \sum_{T \in \mathcal{T}_h(\Omega)} \int_T f(x,y)\phi_i^{(T)}(x,y)dx\,dy + \sum_{e \in \Gamma_{N,h}} \int_e q(s)\phi_i^{(T)}(s)ds \tag{1.5}$$

where e indexes the triangle edges that belong to the Neumann boundary $\Gamma_{N,h}$. From equation (1.4) we see that matrix A is symmetric and sparse. Using Poincarè inequality we can show that it is also positive definite. Hence, the use of sparse linear algebra algorithms is especially recommended.

2 The code

2.1 Definition of the computational mesh

Consider a subdivision of the domain Ω in triangular elements. The triangulation is identified by a unique numbering of the nodes (the vertices of the triangles) and by defining each element via a table that lists (in counterclockwise order) the indices of the triangle nodes. In general, for each triangle an extra integer is added to define the material zone to which the triangle belongs. Finally every node is determined by its coordinates x and y with respect to a common Cartesian reference system.
It is necessary to identify the portions of the domain boundary characterized by Dirichlet or Neumann conditions. This is achieved by specifying the set of nodes that belong to Γ_D or Γ_N.

Algorithm 1 Calculation of local stiffness matrix (coefficients of basis functions for 2D $\mathcal{P}_1$ Galerkin).

```
1: procedure COEFF(Nel,Triang,xy)
   % Linear 2D basis function: phi(xy)=(a+bx+cy)/2A
   % Nel=number of elements
   % Triang=elementwise local-to-global vertex mapping
   % xy=node coordinates
   % Aloc,Bloc,Cloc=coefficients of linear basis functions
2:     Aloc:=0; Bloc:=0; Cloc:=0
3:     for el=1,Nel do
4:         nodes=Triang(el,:) Area(el)=AREADET(xy(nodes,:))
5:         for i=1,3 do
6:             N1loc=MOD_N(i+1,3)
7:             N2loc=MOD_N(i+2,3)
8:             N1glob=nodes(N1loc)
9:             N2glob=nodes(N2loc)
10:            Aloc(el,i)=xy(N1glob,1)*xy(N2glob,2) -xy(N2glob,1)*xy(N1glob,2)
11:            Bloc(el,i)=xy(N1glob,2)-xy(N2glob,2)
12:            Cloc(el,i)=xy(N2glob,1)-xy(N1glob,1)
13:        end for
14:    end for
15: end procedure
   % Calculate triangle area
16: function AREADET(xy)
17:    AreaDet=...
18: end function
   % Modified MOD (remainder)
19: function MOD_N(i,j)
20:    if mod(i,j)=0 then
21:        remainder=j
22:    else
23:        remainder=mod(i,j)
24:    end if
25: end function
```

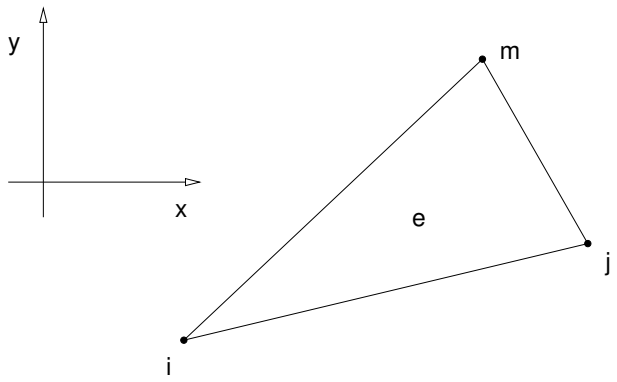


FIGURE 2.1: *Triangular element i, j, m.*

2.2 Evaluation of elemental matrices and their assembly into the final global matrix

Let the domain Ω be subdivided into triangular elements T and let $u_h^{(T)}$ be the numerical solution restricted to element T:

$$u_h^{(T)}(x,y) = \sum_{k=1}^{3} u_k \phi_k^{(T)}(x,y) \tag{2.1}$$

where the summation index k refers to a local (to the element) numbering system. Looking at Figure 2.1, $u_h^{(T)}$ varies linearly between the values of the nodal unknowns u_i, u_j, u_m, that represent the values of the solution u_h on nodes i, j, m. The basis functions $\phi_k^{(T)}$, in fact, are evaluated using a linear Lagrangian interpolation and are thus such that:

- $\phi_k^{(T)}(x)$ assumes nonzero values only inside the element T (they have local support);
- $\phi_k^{(T)}(x) = 1$ on node k, and zero on the other two nodes of T.

They can be given the following expression:

$$\begin{aligned}
\phi_i^{(T)}(x,y) &= (a_i + b_i x + c_i y)/2\Delta \\
\phi_j^{(T)}(x,y) &= (a_j + b_j x + c_j y)/2\Delta \\
\phi_m^{(T)}(x,y) &= (a_m + b_m x + c_m y)/2\Delta
\end{aligned} \tag{2.2}$$

where Δ is the area of T given by

$$\Delta = \frac{1}{2} \det \begin{bmatrix} 1 & x_i & y_i \\ 1 & x_j & y_j \\ 1 & x_m & y_m \end{bmatrix}.$$

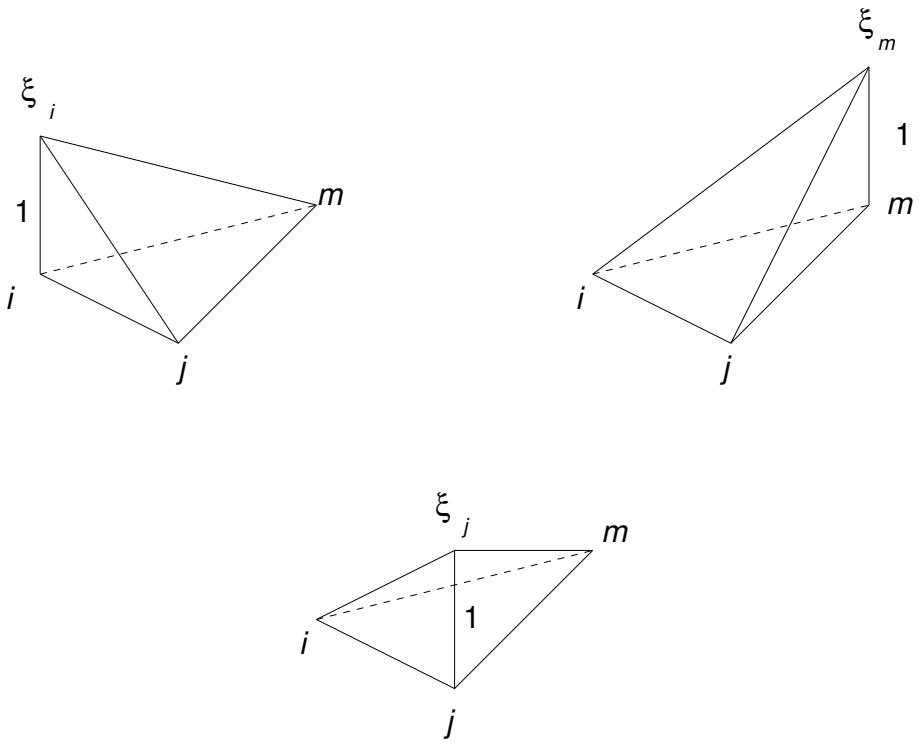


FIGURE 2.2: *Linear basis functions defined on a general triangle with vertices i, j, m.*

Note that it can be shown that $\Delta > 0$ if the nodal numbering in T is counterclockwise. It is convenient, to avoid potenital pitfalls, to take the absolute value of the determinant. Solving the linear system (e.g., with Kramer's rulw) we find that the coefficients a_i, b_i and c_i are given by:

$$\begin{aligned} a_i &= x_j y_m - x_m y_j \\ b_i &= y_j - y_m \\ c_i &= x_m - x_j \end{aligned}$$

where the other coefficients are obtained using a counterclockwise permutation of the indices:

$$\begin{aligned} a_j &= x_m y_i - x_i y_m & b_j &= y_m - y_i & c_j &= x_i - x_m \\ a_m &= x_i y_j - x_j y_i & b_m &= y_i - y_j & c_m &= x_j - x_i \end{aligned}$$

Figure 2.2 shows an example of the basis function $\phi_k^{(T)}$, while Algorithm 1 reports the algorithm for the calculation of he 3×3 local stiffness matrices.
The generic element of the local FEM system matrix can be calculated from (1.4) and is given by:

$$a_{ij}^{(T)} = \int_T \left[\frac{\partial \phi_j^{(T)}}{\partial x} \frac{\partial \phi_i^{(T)}}{\partial x} + \frac{\partial \phi_j^{(T)}}{\partial y} \frac{\partial \phi_i^{(T)}}{\partial y} \right] dx\, dy = \frac{1}{4\Delta} \left(b_i b_j + c_i c_j \right) \tag{2.3}$$

Algorithm 2 Assembly of the stiffness matrix from local contributions (2D $\mathcal{P}_1$ Galerkin).

```
1: procedure STIFF_ASSEMBLY(Nel,Triang,Sloc)
   % Nel=number of elements
   % Triang=elementwise local-to-global vertex mapping
   % Sloc=elementwise (3×3) local stiffness matrix
   % Stiff=global stiffness matrix
2:     Stiff:=0
3:     for el=1,Nel do
4:         for i=1,3 do
5:             iglob=Triang(el,i)
6:             for j=1,3 do
7:                 jglob=Triang(el,j)
8:                 Stiff(iglob,jglob)=Stiff(iglob,jglob) + Sloc(i,j)
9:             end for
10:        end for
11:    end for
12: end procedure
```

Thus the local stiffness matrix is a 3×3 matrix taking on the form:

$$A^{(T)} = \frac{1}{4\Delta}\left\{ \begin{bmatrix} b_i b_i & b_i b_j & b_i b_m \\ b_j b_i & b_j b_j & b_j b_m \\ b_m b_i & b_m b_j & b_m b_m \end{bmatrix} + \begin{bmatrix} c_i c_i & c_i c_j & c_i c_m \\ c_j c_i & c_j c_j & c_j c_m \\ c_m c_i & c_m c_j & c_m c_m \end{bmatrix} \right\} \tag{2.4}$$

The local contributions (2.4) need to be assembled into the global system matrix.

2.3 Evaluation of the right hand side vector

The local contribution (1.5) of the forcing function f to (1.3) for an internal node i can be evaluated via the trapezoidal rule to yield:

$$b_i^{(T)} = \int_T f(x,y)\phi_i^{(T)}(x,y)dx\,dy \approx \frac{|T|}{3}\sum_{j=1}^{3} f(\xi^{(j)})\phi_i^{(T)}(\xi^{(j)}) = \frac{|T|}{3} f(\xi^{(i)}). \tag{2.5}$$

The value of the right-hand-side vector b is then calculated via the same elementwise assembly process used for the stiffness matrix. An alternative more efficient way to do the calculations, is to calculate the nodal area A_i given by the sum of one third of the area of all the triangles sharing node i. This can be obtained again by a simple assembly from the elemental areas. Then, we can write:

$$b_i = -\frac{5}{12}\text{sen}\,(\pi x_i)\cos\left(\frac{\pi}{2}y_i\right) A_i \tag{2.6}$$

For the nodes on the Neumann boundary Γ_N the expression for the local contribution is:

$$b_i^{(e)} = \int_{\Omega^{(e)}} f\phi_i^{(e)} \, \mathrm{d}s + \int_{\Gamma_N^{(e)}} q\phi_i^{(e)} \, \mathrm{d}s \tag{2.7}$$

where now e is the boundary edge of a boundary triangle. Note that in our case the forcing function vanishes at the boundary of Ω and thus only the second term needs to be evaluated This integral can be evaluated again using the trapezoidal rule. In our case, the calculation simplifies by noticing that nodal contribution of the integral from the trapezoidal rule in node i is equal to the value of the Neumann flux $q_N(x_i)$ multiplied by the length l_i, given by the sum of one half of the length of the edges sharing node i:

$$b_i = \frac{1}{2} q(y_i) l_i = \frac{\pi}{2} \mathrm{sign}(x_i) \cos\left(\frac{\pi}{2} y_i\right) l_i \tag{2.8}$$

Thus, the Neumann condition is implemented by adding to the right-hand-side vector the contribution of the Neumann flux evaluated on node i and multiplied by the arithmetic average l_i of the lengths of the two edges sharing node i.

2.4 Imposing the Dirichlet boundary conditions

Dirichlet boundary conditions can be imposed either using the boundary lifting approach or the penalty method with a penalty value $eps = 10^{-15}$.

2.5 Solution of the linear system

The linear system (1.3) can be solved by means of the Matlab backslash operator or, if one wishes, by the Preconditioned Conjugate gradient algorithm, as implemented in Matlab and using the sparse linear algebra facility. The typical preconditioner is the so called Incomplete Choleski decomposition with zero fill-in.

3 Application of the code

Develop a FEM code for the solution of the previously described problem and compare the obtained numerical solution with the given analytical solution (1.2). To verify the correctness of the code, we want to determine its experimental convergence and comparing it with the theoretical provisions. To this aim, solve the problem on the sequence of uniformly refined meshes provided by the lecturer, and evaluate the norm of the error at each refinement level. Every successive refinement is obtained, as shown in Figure 3.1, by connecting each edge midpoint.

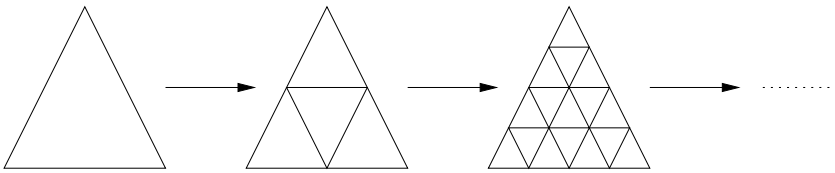


FIGURE 3.1: *Mesh uniform refinement scheme.*

Evaluate the L_2 norm of the error by means again of the trapezoidal rule to estimate:

$$\varepsilon_h = \left[\int_\Omega (u_h - u)^2 \, dx \right]^{1/2}, \tag{3.1}$$

where u_h is the solution evaluated using the FEM code, and u is the real solution given by (1.2). The integral is approximated again by the trapezoidal rule.
Use the triangulations (mesh0, mesh1, mesh2, mesh 3, mesh4) provided in the project material and solve the problem on each of them, evaluating the error and the experimental convergence rate from the sequence of errors.
The student is required to write a short report describing the experience acquired and the steps performed. In particular, the report should contain:

- FEM convergence table reporting for every refinement level (i) the value ε_h the error norm, the ratio between the errors at the current and previous refinement levels. Add the convergence plot of the error as a function of the mesh parameter in log-log scale;
- contour or surface plots of the solution (numerical and real) at a reasonable mesh refinement;
- contour or surface plot of the error function $(u - u_h)$ at the same refinement level;
- PCG convergence plots (optional) in semi-log scale of the conjugate gradient iteration for all the five triangulations. The plots should display the Log10 of the relative norm of the PCG residual r_k vs. the iteration number k; for some cases of your choice, display also the convergence behavior of the PCG residual in comparison with the real residual $r_k = b - Ax_k$.

A The Matlab templates and the mesh definitions

A template (or skeleton) of a 2D FEM matlab script is provided (available in the moodle site at https://stem.elearning.unipd.it/pluginfile.php/1351416/mod_folder/content/0/poisson.zip). Algorithm 3 shows the template of the main Matlab script that can be used as a guide to build the working FEM code. The template provides an input data routine, fully implemented and

Algorithm 3 Template of 2D FEM P1 code. Main Matlab script

```
% Finite element code for 2D piecewise linear Galekrin
% -div(Diff grad u)(x)=f(x) with Dirichlet BC
%
% Define the 2D mesh and equation BC's and coefficients
meshdir='mesh1';
[Nelem,Nodes,triang,coord,NDir,DirNod,DirVal,ureal,force]= input_data(meshdir);
%
% calculate element area and elemental coefficients of basis functions
% (or its gradient (b,c)) phi(x,y)=a+bx+cy
% [Bloc,Cloc,Area]=localBasis(Nelem,triang,coord);
%
% build stiffness matrix (without BCs)
% stiffMat=stiffBuild(Nelem,Nodes,triang,Bloc,Cloc,Area,Diff);
%
% build convection matrix (without BCs)
% convMat=convBuild(Nelem,Nodes,triang,Bloc,Cloc,Area,beta);
%
% build stabilization matrix (without BCs)
% stabMat=stabBuild(Nelem,Nodes,triang,Bloc,Cloc,Area,delta);
%
% sysMat=stiffMat+convMat+stabMat;
%
% impose BCs
% [sysMat,rhs]=imposeBC(Nodes,NDir,DirNod,DirVal,sysMat);
% Solve linear sysyem
% uh=sysMat\rhs;
ur=ureal(coord(:,1),coord(:,2));
figure(1);
trisurf(triang,coord(:,1),coord(:,2),ur);
figure(2);
plot_tri(coord,triang,1,'node_index_l')
plot_tri(coord,triang,1,'edges')
plot_tri(coord,triang,1,'face_index')
```

Algorithm 4 Template of 2D FEM P1 code. "input_data" Matlab function

```
1: function [Nelem,Nodes,triang,coord,NDir,DirNod,DirVal,ureal,force]=
2: input_data(meshdir)
3: meshtriang=strcat(meshdir, '/triang.dat');
4: meshcoord=strcat(meshdir, '/xy.dat');
5: meshDirNod=strcat(meshdir, '/dirtot.dat');
   % meshDirVal=strcat(meshdir, '/dirval.dat');
6: load(meshtriang);
7: load(meshcoord);
8: load(meshDirNod);
   % load(meshDirVal);
9: DirVal=0;
10: triang=triang(:,1:3);
11: DirNod=dirtot';
   % DirVal=dirval';
   %
12: Nelem=size(triang,1);
13: Nodes=size(coord,1);
14: NDir=size(DirNod,1);
   %
15: ureal=@(x,y) sin(pi.*x).*cos(pi.*y/2);
16: force=@(x,y) 5*pi*ureal(x,y)/4;
17: end
```

described below, and the calls (commented) to the functions to be implemented, including some functions for project 2 (convection + stabilization).
Line 1 stores in the variable `meshdir` the directory where the data files are located. There are 5 directories named `mesh0`, `mesh1`, `mesh2`, `mesh3`, `mesh4`, that contain the files defining the mesh (element connectivity, coordinates, boundary nodes, Dirichlet boundary values). Each directory contains the following files:

- `triang.dat` the element-node connectivity array
- `xy.dat` the coordinates of the nodes
- `dirnod.dat` the indices of the Dirichlet boundary nodes
- `neunod.dat` the indices of the Neumann boundary nodes
- `dirtot.dat` the indices of all the boundary nodes

Algorithm 4 contains the `input_data.m` matlab function. This matlab function reads in the variable `meshdir` containig the name of the directory where the files are stored, and concatenates it with the string containing the name of the file with the data to be read. For example Line 3 concatenates the directory name (e.g., `mesh0`) with the string `/triang.dat` obtaining `mesh0/triang.dat`. Then line 6 is used to stores the content of the file identified by the string into the variable `triang`, i.e. the filename without any extension. Line 10 is used to remove the last column in the input array `triang` which contains useless "ones". The file `dirtot.dat` can be used to impose Dirichlet boundary conditions on the entire boundary, noting that homogeneous Dirichlet is a condition that leads to the same solution (1.2). This file must be replaced by `dirnod.dat` and `neunod.dat` for the final project to read in the appropriate Dirichlet and Neumann boundaries. Obviously, the code must be adapted appropriately following the same construction described above for `triang.dat`.
It is suggested that at the beginning the development considers only homogeneous Dirichlet so that one does not have to think about implementing Neumann boundary conditions, and one can check that the code achieves the sought convergence rates without worrying about possible bugs in the implementation of non-homogeneous Neumann boundaries.
The calculation of the Neumann flux for the latter is done using eq. (2.8). Here we need the value of l_i, i.e., the arithmetic average of the lengths of the boundary edges sharing node i. We note that for `mesh0` this value is always $1/2$ and then it is halved for each successive mesh. It is suggested that one uses this simple strategy, as the calculation of l_i is not straight forward in terms of implementation.

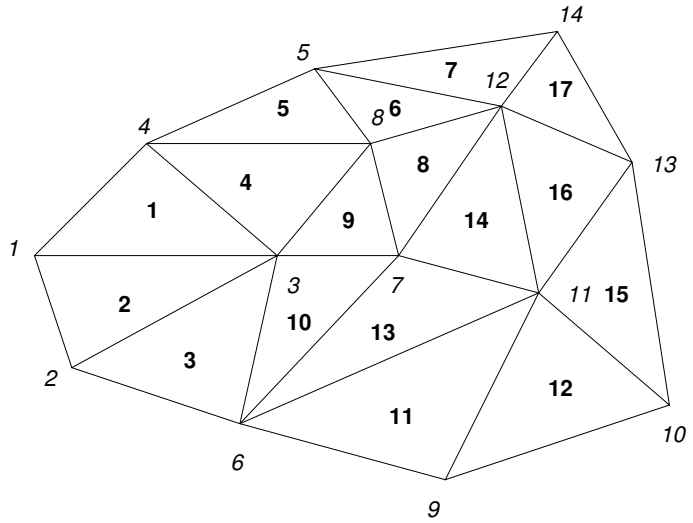


FIGURE B.1: *Triangular element patch. Italic characters indicate node indices, while bold face characters denote element indices.*

B Assembly of the stiffness matrix from the local contributions

The process of building the matrix A is called "assembly" process and starts by defining local elementwise matrices (generally not stored) and build the global contributions by summing up these pieces. Formally we usually write:

$$a_{ij} = \sum_{T \in \mathcal{T}_h(\Omega)} a_{ij}^{(T)}$$

where matrix $A^{(T)}$ is of dimension $n^{(T)} \times n^{(T)}$, where $n^{(T)}$ is the number of nodes in the element. For example, for a triangular grid with 3 nodes for each triangle the local element stiffness matrix is of dimension 3×3, i.e., $a_{ij}^{(T)}$, $i = 1, 2, 3$, $j = 1, 2, 3$.

For example, consider a triangle of nodes i, j, m, given in this order in the table stored in array `TRIANG`. Then element $a_{11}^{(T)}$ of the local matrix must be summed up to element $a_{ii}^{(T)}$ of the global element, since i corresponds to the first node in `TRIANG`, while the local element $a_{12}^{(T)}$ will be summed up to the global element a_{ij}, and so forth.

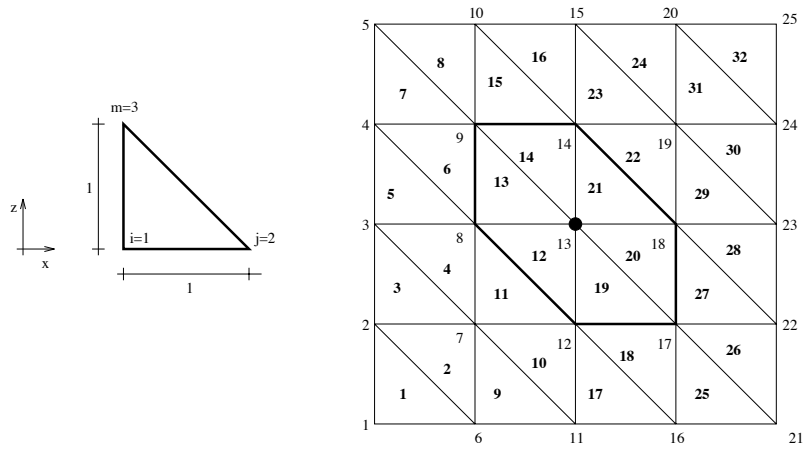
To exemplify the procedure, let us look again at the example given in Figure B.1. We want to build the contribution of element **13** to the global matrix. Element **13** is characterized by the nodes *7, 6, 11*. We have thus the following connections:

	7	6	11
7	$a_{1,1}^{(13)}$	$a_{1,2}^{(13)}$	$a_{1,3}^{(13)}$
6	$a_{2,1}^{(13)}$	$a_{2,2}^{(13)}$	$a_{2,3}^{(13)}$
11	$a_{3,1}^{(13)}$	$a_{3,2}^{(13)}$	$a_{3,3}^{(13)}$

Thus, $a_{1,1}^{(13)}$ will be summed to element $a_{7,7}$, $a_{1,2}^{(13)}$ to element $a_{7,6}$, $a_{1,3}^{(13)}$ to element $a_{7,11}$, and so fort. Note that we can exploit the symmetry of A and work only on its upper triangular portion. Hence, for example, the diagonal element $a_{7,7}$ of the global matrix will be formed by summing all the local contributions from the elements that have node 7 in common, in this case, elements **8**, **9**, **10**, **13**, and **14**, which are described in the array `TRIANG` as follows:

```
...
 8   7  12
 8   3   7
 3   6   7
...
 7   6  11
 7  11  12
...
```

Thus, $a_{7,7}$ will be given by $a_{7,7} = a_{2,2}^{(8)} + a_{3,3}^{(9)} + a_{3,3}^{(10)} + a_{1,1}^{(13)} + a_{1,1}^{(14)}$ On the other hand, the non-diagonal global element $a_{7,11}$ will receive the contributions of the triangles that have in common the edge $7 - 11$, i.e., triangles **13** and **14**. Thus we will have $a_{7,11} = a_{1,3}^{(13)} + a_{1,2}^{(14)}$.



e =**19**						12	17	13	
	e =**20**					18	13	17	
		e =**21**				13	18	14	
			e =**14**			14	9	13	
				e =**13**		8	13	9	
					e =**12**	13	8	12	
12	18	13	14	8	13	$h^e_{1\,1}$	$h^e_{1\,2}$	$h^e_{1\,3}$	
17	13	18	9	13	8	$h^e_{2\,1}$	$h^e_{2\,2}$	$h^e_{2\,3}$	
13	17	14	13	9	12	$h^e_{3\,1}$	$h^e_{3\,2}$	$h^e_{3\,3}$	

	8	9	12	13	14	17	18
19			$h^{19}_{3\,1}$	$h^{19}_{3\,3}$		$h^{19}_{3\,2}$	
20				$h^{20}_{2\,2}$		$h^{20}_{2\,3}$	$h^{20}_{2\,1}$
21				$h^{21}_{1\,1}$	$h^{21}_{1\,3}$		$h^{21}_{1\,2}$
14		$h^{14}_{3\,2}$		$h^{14}_{3\,3}$	$h^{14}_{3\,1}$		
13	$h^{13}_{2\,1}$	$h^{13}_{2\,3}$		$h^{13}_{2\,2}$			
12	$h^{12}_{1\,2}$		$h^{12}_{1\,3}$	$h^{12}_{1\,1}$			
	$h^{12}_{1\,2}+h^{13}_{2\,1}$	$h^{13}_{2\,3}+h^{14}_{3\,2}$	$h^{12}_{1\,3}+h^{19}_{3\,1}$	$h^{12}_{1\,1}+h^{13}_{2\,2}+h^{14}_{3\,3}+$ $h^{21}_{1\,1}+h^{20}_{2\,2}+h^{19}_{3\,3}$	$h^{14}_{3\,1}+h^{21}_{1\,3}$	$h^{20}_{2\,3}+h^{19}_{3\,2}$	$h^{21}_{1\,2}+h^{20}_{2\,1}$

FIGURE B.2: *Another example of assembly from the local to the global stiffness matrix*